2 Rust Implementation

Abstraction	Cumulative responsibilities
Module	• Represent and allows the execution of an individual parameterized transformation $g_l(z_{l-1}; \theta_l)$ as an element of an arbitrarily long composition; • provide, under the adopted *define-and-run* strategy, an ordered, inspectable representation of the composition that is traversable in reverse and maintains the association between each transformation g_l, its parameter block θ_l, and the preceding computation; • during the forward pass, retain the intermediate values z_l required by the corresponding backward step; • implement internally the VJP of its own transformation, $$\mathrm{VJP}_{g_l} : (\theta_l, z_{l-1}, a_l) \longmapsto (a_{l-1}, \nabla_{\theta_l} \ell),$$ thereby computing both the local parameter gradient and the adjoint to be propagated to the preceding step; • expose the resulting $(\theta_l, \nabla_{\theta_l} \ell)$ pairs to the external optimization mechanism, without applying parameter updates itself.
Tensor	Provide a representation for the numerical values exchanged between successive transformations, such that the output of each transformation is admissible as the input of the next one.
Loss	Provide a dedicated abstraction, external to the chain of Modules, for the terminal objective evaluation. Given the final network output z_L and the target y, compute both the scalar sample-wise loss $$\ell(z_L, y) \in \mathbb{R}_{\geq 0}$$ and the seed of the reverse accumulation, $$a_L = \nabla_{z_L} \ell(z_L, y).$$ The produced adjoint must have the same shape as a_l for each admissible l, so that it is a valid input adjoint for the VJP of the final Module.
Optimizer	• Consume the per-example parameter gradients exposed by the Module mechanism and accumulate them, if necessary, across the examples of a batch to obtain the batch gradient $$\nabla_{\theta_l} \mathcal{L}_{\mathcal{B}} = \frac{1}{\lvert\mathcal{B}\rvert} \sum_{i \in \mathcal{B}} \nabla_{\theta_l} \ell_i$$ for every parameter block θ_l; • It has to be possible to access to $(\theta_l, \nabla_{\theta_l} \mathcal{L}_{\mathcal{B}})$ for $l = 1, \ldots, L$ • apply a parameterized update rule to each parameter block and maintain any auxiliary state required by that rule, with the state decomposed as $s = (s_1, \ldots, s_L)$ and with $s_l = \emptyset$ for stateless methods such as SGD.

Table 1: Cumulative responsibilities of the four abstractions derived from the structural, differentiation, and optimization requirements.

2.1 Design of the `Module` Abstraction

In order to implement the `Module` abstraction, we must first translate into code the semantic requirements established in the previous chapter.

Building on these, our initial formulation of composition required nothing more than the existence of a finite mechanism, not indexed by L: an abstract constraint that, as we noted at that point, did not in itself impose a single, uniform interface for each transformation. However, the progressive introduction of additional semantic requirements, together with the extensions they necessitated, substantially refines this original formulation.

The mechanism must guarantee an ordered, inspectable, and backward-traversable representation of the composition, maintaining the univocal association between the transformation g_l and its parameters θ_l. To this must be added the need to retain, during the forward pass, the intermediate values z_l required by the corresponding backward step, the internal implementation of the VJP, and the re-exposure of θ_l and $\nabla_{\theta_l}\ell$. Taken together, these requirements turn semantic uniformity from a mere design choice into a necessary architectural constraint.

Without a single operational contract, in fact, both the execution of the backward pass, in propagating the gradient a_l through the various transformations, and the optimizer's update cycle would be forced to handle ad hoc cases for each specific layer type. This would introduce a branching of checks that would destroy the linearity of the chain, increasing code complexity and defeating the very idea of generic composition.

It is precisely from this necessity that the challenge shifts to the implementation level: this semantic uniformity must now be translated concretely into Rust's type system. This brings us to the central question of the present section: *how can we guarantee, in Rust, the semantic uniformity of the interface over a collection of elements that are physically heterogeneous in memory?*

2.1.1 Polymorphism as a Foundation for Interface Uniformity

In order to translate the notion of semantic uniformity into an operational construct, we rely on a concept introduced during the course: *polymorphism*.
As we have seen in class, the standard taxonomy distinguishes two broad families: *ad hoc polymorphism* and *universal polymorphism*.

Ad hoc polymorphism comprises two principal forms:

- **Overriding**: a mechanism whereby a subclass redefines a method inherited from a superclass, with the specific implementation selected at runtime via dynamic binding based on the receiver's dynamic type. Within the type system hierarchy, overriding introduces *ad hoc* polymorphism within the context of inheritance-based universal polymorphism. Since this mechanism is intrinsically linked to class-based inheritance, languages lacking traditional object-oriented inheritance, such as Rust, do not implement this form of polymorphism.
- **Overloading**, an ad hoc polymorphism mechanism whereby multiple functions or operators share the same name but have different signatures. The correct implementation is selected on the basis of the static types of the arguments, resulting in early (static) binding in statically typed languages. While classical overloading is common in languages like Java and C++, Haskell and its descendant Rust handle these concepts cleanly through type classes and traits rather than traditional ad hoc function overloading.

Neither of these two forms, however, is what is required here.
What we need is *universal polymorphism*: the ability to provide a single, universal solution where only one algorithm applies to objects of different types. Unlike ad hoc polymorphism, where the same function name denotes different algorithms determined by the actual types, universal polymorphism allows a single piece of code to operate uniformly across multiple types without needing specialized implementations for each.
Universal polymorphism itself admits two realizations, *parametric polymorphism* and *inclusion polymorphism*, which we examine below as candidate mechanisms for composing layers.

2.1.2 The `Module` Trait

The natural vehicle for expressing this uniformity in Rust is the `trait` construct. As we have seen in class, a trait can be understood as a formal contract: a named set of method signatures that a type must implement in order to be considered a member of the trait, without the contract itself prescribing anything about how the implementing type is laid out in memory or allocated.
Under this view, every layer of the network is required to provide an implementation of a common `Module` trait.

A first, non-rigorous but sufficiently concrete formulation of this trait is the following:

```
1 pub trait Module {
2     /// Computes the output tensor z_l from the input tensor z_{l-1}.
3     fn forward(&mut self, input: &Tensor) -> Tensor;
4
```

```
    /// Given the upstream gradient a_l = d loss / d z_l,
    /// returns the downstream gradient a_{l-1} = d loss / d z_{l-1}.
    fn backward(&mut self, grad_output: &Tensor) -> Tensor;

    /// Exposes read-only access to the layer's parameters theta_l.
    fn params(&self) -> Vec<&Tensor>;

    /// Exposes the gradients w.r.t. the parameters, grad theta_l L.
    fn grads(&self) -> Vec<&Tensor>;
}
```

Here `Tensor` denotes, for the time being, an unspecified data structure; its concrete implementation will be addressed later. The ownership model adopted by this interface should likewise be regarded as provisional and will be refined in a later section.
The signature of `forward` reflects the requirement that a transformation g_l map an input tensor to an output tensor; `backward` reflects the requirement that the layer be able to propagate a gradient a_l backward into a_{l-1}, using whatever intermediate state z_l it retained during the forward pass; `params` and `grads` reflect the requirement to re-expose θ_l and $\nabla_{\theta_l}\ell$ to the optimizer.

2.1.3 The Problem of Heterogeneous Composition

Once the `Module` trait has been defined, if only informally, the next question is:
how a sequence of layers is to be composed.
Naively, one might attempt to store all layers in a single, homogeneous collection such as `Vec<Module>`. This is, however, not directly possible: `Module` is a trait, not a concrete type, and a `Vec<T>` requires `T` to be a single, statically known type of fixed size. Each concrete struct implementing `Module` (a linear layer, an activation, and so on) will in general have a different size and internal layout, so no single `T` can represent all of them. We are therefore confronted with a genuine requirement for *heterogeneity*: the collection must be able to hold values of different concrete types, all satisfying the same interface.

Two mechanisms are available in Rust to address this requirement, corresponding to the two forms of universal polymorphism introduced above.

2.1.4 Parametric Polymorphism: Composition via Generics

The first approach is to use *parametric polymorphism*, i.e., Rust's generic type parameters, to build composition structurally rather than through a homogeneous collection. Two modules can be chained together by means of a generic wrapper struct:

```
pub struct Chain<A, B> {
    pub first: A,
    pub second: B,
}

impl<A: Module, B: Module> Module for Chain<A, B> {
    fn forward(&mut self, input: &Tensor) -> Tensor {
        let z = self.first.forward(input);
        self.second.forward(&z)
    }

    fn backward(&mut self, grad_output: &Tensor) -> Tensor {
        let a = self.second.backward(grad_output);
        self.first.backward(&a)
    }

    fn params(&self) -> Vec<&Tensor> {
        let mut p = self.first.params();
        p.extend(self.second.params());
        p
    }

    fn grads(&self) -> Vec<&Tensor> {
        let mut g = self.first.grads();
        g.extend(self.second.grads());
        g
    }
}
```

The key observation is that, whenever `A` and `B` both implement `Module`, `Chain<A, B>` itself implements `Module`; the construction is therefore closed under composition, and arbitrarily long chains can in principle be built by nesting, e.g., `Chain<A, Chain<B, C>>`.

The advantages of this approach follow directly from the fact that Rust generics are a *zero-cost abstraction*: at compile time, the compiler *monomorphizes* each distinct instantiation of `Chain`, generating specialized, statically dispatched code with no run-time indirection.

This makes aggressive inlining of `forward` and `backward` across layer boundaries possible, and, since the size and layout of every `Chain<A, B>` instance is known at compile time, the whole composed structure can be allocated contiguously (e.g., on the stack), which in turn improves data locality and cache behavior relative to a heap-allocated, pointer-chasing representation.

The central limitation, however, is that the *type* of the composed network encodes its entire structure: a network of L layers has a type of nesting depth L, e.g., `Chain<A, Chain<B, Chain<C, D>>>`. This has two undesirable consequences.

First, the type quickly becomes unwieldy and effectively prevents building networks whose depth or structure is determined at run time (e.g., read from a configuration file), since the type of the resulting composition would have to be known at compile time.

Second, and more importantly for the architecture as a whole, abstractions that operate over the entire network become coupled to its exact nested type.Consider a simple `Optimizer`, the implementation of which will be discussed in subsequent sections: although it depends exclusively on the `Module` interface, it must nonetheless be generic with respect to the model's concrete type:

```
pub trait Optimizer<M: Module> {
    fn step(&mut self, model: &mut M);
}

pub struct Sgd<M: Module> {
    // fields omitted, not yet specified
}

impl<M: Module> Optimizer<M> for Sgd<M> {
    fn step(&mut self, model: &mut M) { /* ... */ }
}

// The full nested type must still be spelled out here:
let model: Chain<Linear, Chain<ReLU, Chain<Linear, Softmax>>> = /* ... */;

let mut opt: Sgd<Chain<Linear, Chain<ReLU, Chain<Linear, Softmax>>>> = /* ... */;
```

Thus, adding, removing, or reordering a single layer changes the model type and consequently the type of every abstraction parameterized by it, such as the optimizer, rather than allowing them to depend solely on the `Module` interface.

Moreover, while monomorphization does yield measurable performance benefits, in the setting of deep learning the actual computational bottleneck lies elsewhere, namely in the dense matrix operations performed inside each layer, so the zero-cost guarantee offered by this approach addresses a cost that is not the dominant one in practice.

2.1.5 Inclusion Polymorphism: Composition via Trait Objects

The second approach to heterogeneous `Module` implementations is *inclusion polymorphism* via Rust *trait objects*. Unlike statically resolved generics like `Chain<A, B>`, a trait object of type `dyn Module` erases the concrete type, deferring method resolution to runtime.

Because Rust requires every stack variable to have a statically known size, and a `dyn Module` has an unknown, variable size (e.g., a `Linear` layer takes 200 bytes while a `ReLU` takes 0), it is classified as a *dynamically sized type* (DST). Consequently, a bare declaration like `let x: dyn Module` is rejected by the compiler.

To bypass this stack limitation, the concrete value is allocated on the heap while its size is still known, leaving only a fixed-size reference on the stack. This is precisely what `Box<dyn Module>` provides: a stack-resident handle owning a heap-allocated value. This alone, however, only solves the *size* problem, not the *dispatch* problem: a raw data pointer is insufficient because a runtime call like `layer.forward(x)` still cannot tell, from the pointer alone, which underlying type, and hence which implementation of `forward`, it is addressing.

To solve this, Rust employs a *fat pointer* spanning 16 bytes on 64-bit architectures, combining two elements:

1. A **data pointer** pointing to the heap-allocated object.
2. A **vtable pointer** referencing a static, precompiled table of function pointers for that specific concrete type.

This uniform 16-byte representation enables heterogeneous collections such as `Vec<Box<dyn Module>>`, where elements vary in type and size but share an identical stack footprint.

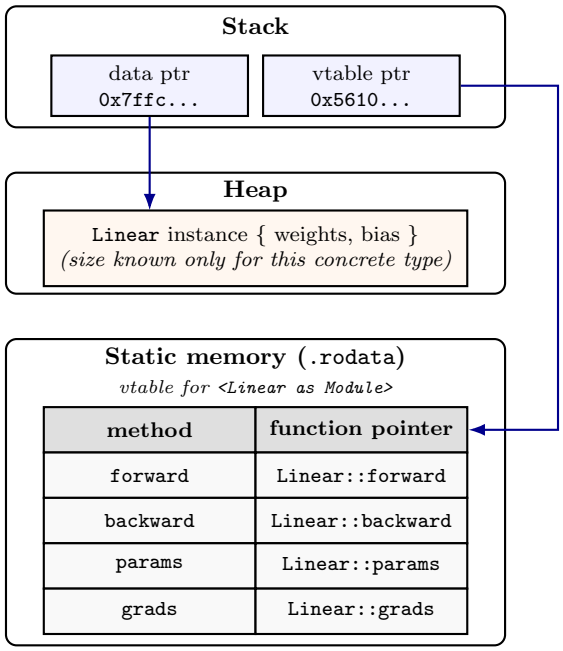

Figure 3: Runtime layout of a `Box<dyn Module>` holding a `Linear` value, read top to bottom. The *stack* holds only the fat pointer itself: two machine words of fixed total size (16 bytes), regardless of the concrete type behind them. The *data pointer* addresses the actual `Linear` instance on the *heap*, whose size is known only because the concrete type is still visible at the point of allocation. The *vtable pointer* addresses a fixed-size table in *static memory*, generated once per concrete type at compile time, mapping each method of `Module` to the concrete function implementing it for `Linear`. A call to `layer.forward(x)` follows the vtable pointer, looks up the `forward` row, and invokes the function pointer found there with the data pointer supplied as the receiver (`self`).

There is a detail we have quietly skipped over: not every trait can be turned into a trait object. The vtable mechanism, as just described, only works if the trait's methods satisfy two conditions. `Module` happens to satisfy both, which is exactly why `Box<dyn Module>` was usable above without any complication, but it is worth seeing why, since the construction would fail to compile otherwise.

First constraint: no method may introduce an extra generic type parameter. A single vtable entry is one function pointer to one piece of compiled code, but a generic method compiles to a *different* piece of code for every type it is called with (monomorphization). In principle, if the compiler saw the entire program at once, it could just count the used types and build a custom vtable. In reality, imagine publishing a library: if someone uses it tomorrow with a brand-new type, the compiler cannot retroactively add a new slot to your vtable. One might argue: *"Just recompile the library from source!"* But if Rust rebuilt vtables based on global usage, a trait's memory layout and validity would constantly shift depending on function calls written thousands of lines away. Rust strictly enforces local reasoning: a trait's contract must be predictable and fixed from its definition alone. Therefore, reserving a finite, stable number of slots for a generic method is impossible.

```
trait Module {
    fn load_into<T>(&self, target: &mut T); // rejected: dyn Module is illegal
    fn forward(&self, x: &Tensor) -> Tensor; // fine: no type parameters
}
```

`Module`, as actually written, has no method like `load_into`: `forward`, `backward`, `params`, `grads` each compile to exactly one function per concrete type, i.e. exactly one vtable entry.

Second constraint: no method may take or return `Self` by value. Suppose `Module` included a method like `fn duplicate(&self) -> Self`. In principle, returning an object by value simply means copying its bytes into a reserved spot on the caller's stack. But imagine writing a function that relies on dynamic dispatch:

```
fn clone_layer(m: &dyn Module) {
    let copy = m.duplicate(); // How much memory do we allocate for `copy`?
}
```

Here is the physical roadblock: the compiler must reserve stack space for `copy` *before* the program runs. If the hidden type behind `m` is a `ReLU`, it needs 0 bytes; if it is a complex `Linear` layer, it might need 200 bytes. Because `dyn Module` intentionally erases the original type, the compiler is entirely blind to the concrete struct and cannot possibly guess the required size. `Module`, as written, avoids this trap: its methods only take pointers (`&self` or `&mut self`, which are always the same size) and return fixed-size types like `Tensor`, guaranteeing that the memory layout is universally predictable. Because `Module` satisfies both constraints, `dyn Module` is a legal type, and the collection built above is valid: a common pointer type can store any concrete struct implementing `Module`, regardless of its size or layout.

Sequential Having established that `dyn Module` is a legal trait object, we introduce a concrete type built around `Vec<Box<dyn Module>>`, called `Sequential`. Its role is twofold: own a run-time extensible, heterogeneous list of layers, and itself satisfy the `Module` contract, so that an entire network can be handled from the outside exactly like a single layer, nested inside another `Sequential`, or passed to an `Optimizer` written solely against the `Module` interface.

```
pub struct Sequential {
    layers: Vec<Box<dyn Module>>,
}

impl Sequential {
    pub fn new() -> Self {
        Self { layers: Vec::new() }
    }

    pub fn add(&mut self, layer: Box<dyn Module>) -> &mut Self {
        self.layers.push(layer);
        self
    }
}
```

At this stage `Sequential` is only a growable box of trait objects; it does not yet behave as a `Module`. We establish conformance by explicitly implementing its methods, with each method walking through `self.layers`:

```
impl Module for Sequential {
    fn forward(&mut self, input: &Tensor) -> Tensor {
        let mut current = input.clone();
        for layer in self.layers.iter_mut() {
            current = layer.forward(&current);
        }
        current
    }

    fn backward(&mut self, grad_output: &Tensor) -> Tensor {
        let mut current = grad_output.clone();
        for layer in self.layers.iter_mut().rev() {
            current = layer.backward(&current);
        }
        current
    }

    fn params(&self) -> Vec<&Tensor> {
        self.layers.iter().flat_map(|l| l.params()).collect()
    }

    fn grads(&self) -> Vec<&Tensor> {
        self.layers.iter().flat_map(|l| l.grads()).collect()
    }
}
```

`forward` folds the input through the layers in `add` order, feeding z_l as the input to g_{l+1};
`backward` performs the symmetric traversal via `rev()`, propagating a_l from the last layer to the first.
Both methods accept references (`&Tensor`) in accordance with the provisional definition of the `Module` trait. As we will see in the next section, passing references across sequential layers forces intermediate tensor clones during iteration, providing the direct architectural motivation for refining our ownership model to take tensors by value.
`params` and `grads` use `flat_map` rather than `map` because each `l.params()` already returns a `Vec<&Tensor>`: this flattens the per-layer vectors into one single `Vec<&Tensor>` for the whole network, instead of a nested `Vec<Vec<&Tensor>>`, re-exposing every $(\theta_l, \nabla_{\theta_l}\ell)$ pair to the optimizer regardless of layer type.

With this construction, the design of the `Module` abstraction is complete at the level of interface uniformity: heterogeneous layers are composed into a single, ordered, traversable structure, without sacrificing the semantic contract established at the beginning of this section.

2.2 Design of the `Tensor` Abstraction

Table 1 assigns to `Tensor` a single, apparently modest responsibility: to represent the values exchanged between successive transformations so that the output of g_l is admissible as the input of g_{l+1}. Read literally, this is a statement about the *forward* pass only: it guarantees that z_l can be fed into g_{l+1}. Before turning to how `Tensor` should be represented in Rust, it is worth asking whether this same guarantee extends, for free or otherwise, to the *backward* pass, that is, whether the adjoint a_{l-1} produced by VJP_{g_l} is automatically admissible wherever z_{l-1} was. If it is, a single `Tensor` type suffices for both directions of data flow; if it is not, the design needs a second, dedicated type for adjoints, and every signature in `Module` would have to be duplicated accordingly.

2.2.1 Dimensional Compatibility Along the Chain

Claim. For every admissible l, the adjoint a_l has exactly the same dimension as z_l.

The backward recursion already gives, for $l = L, \dots, 1$,

$$a_{l-1} = \left(\partial g_l / \partial z_{l-1}\right)^\top a_l.$$

Write $J_l := \partial g_l / \partial z_{l-1} \in \mathbb{R}^{n_l \times n_{l-1}}$, where n_{l-1}, n_l denote the dimensions of z_{l-1}, z_l respectively — so J_l's shape is fixed purely by the two vectors it maps between, whatever g_l is. Transposing swaps its two dimensions, $J_l^\top \in \mathbb{R}^{n_{l-1} \times n_l}$, and then

$$a_{l-1} \;=\; \underbrace{J_l^\top}_{n_{l-1} \times n_l} \; \underbrace{a_l}_{n_l} \;\in\; \underbrace{\mathbb{R}^{n_{l-1}}}_{\text{same dimension as } z_{l-1}}.$$

This holds for every l regardless of what g_l actually computes: only the dimensions of $J_l^\top$ matter, not its entries, proving the claim.

Since a_l and z_l always live in the same space, a single `Tensor` type can carry both, licensing the dual signatures of `Module`:

```
fn forward(&mut self, input: &Tensor) -> Tensor;
fn backward(&mut self, grad_output: &Tensor) -> Tensor;
```

A dedicated `Adjoint`/`Gradient` type would add no expressiveness, only duplicate machinery `Tensor` already provides.

2.2.2 Representing a `Tensor` in Rust

With the shape argument settled, `Tensor` can be given a concrete Rust representation.
Following standard practice in dense numerical computing (BLAS-style libraries, NumPy, `ndarray`), the data itself is stored as a single *flat buffer*: one contiguous run of homogeneous scalars, with no structure of its own. All multi-dimensional structure, how many axes there are, how large each is, how to walk from one logical position to the next in memory, is layered *on top of* this buffer as separate metadata, rather than being baked into the storage itself. This separation of "raw storage" from "logical view" is not incidental; it is exactly what will make several operations essentially free in Section 2.2.5.

2.2.3 The `data` Field

A first candidate for the buffer's type is Rust's fixed-size array, `[f32; N]`, whose main attraction is that `N` is known at compile time: the whole tensor could then be allocated contiguously on the stack, with no heap indirection and a memory layout fully known to the compiler ahead of time, the same kind of benefit that made monomorphized `Chain<A,B>` attractive in Section 2.1.

This benefit, however, cannot be realized here, for two independent reasons, each sufficient on its own to rule the array out:

- **The per-example dimensionality depends on the dataset.** An MNIST image contributes $28 \times 28 = 784$ scalars; another dataset contributes some other count. Baking this into the type as a const generic, `Tensor<const N: usize>`, would make `Tensor<784>` and `Tensor<256>` mutually incompatible types, the same "type encodes structure" problem already diagnosed for `Chain<A, Chain<B,C>>`, undoing the interface uniformity that motivated `dyn Module` in the first place.
- **The batch size is a run-time quantity.** A `Tensor` does not hold one example but a *batch* of B examples, and B is chosen by the training loop, not by the definition of the layer. Worse, B is not even constant *within a single run*: whenever the dataset size is not an exact multiple of the configured batch size, the last batch of an epoch is smaller than the others. A type whose size is fixed once and for all at compile time cannot represent two different concrete sizes at different points of the same program, let alone a size chosen at run time in the first place.

Both obstacles disappear once the buffer is a growable, heap-allocated `Vec<f32>`: its length is a run-time value, so a single `Tensor` type can hold any per-example dimensionality and any batch size, including one that changes from call to call.

```
pub struct Tensor {
    data: Vec<f32>,       // flat buffer, length = product of shape
    shape: Vec<usize>,    // see Section 3.4.4 below
    strides: Vec<usize>, // see Section 3.4.5 below
}
```

2.2.4 The `shape` Field

On its own, `data` is a meaningless flat run of n scalars: nothing in a `Vec<f32>` of length $B \cdot 784$ says whether it holds B images of 784 pixels each, 784 signals of B samples each, or something else entirely. The `shape` field, `shape: Vec<usize>`, resolves this ambiguity: it is an ordered list of extents, one per axis, that tells the buffer how to be grouped and nested, equivalently, it fixes the rank of the tensor (its length) and, via $n = \prod_k \texttt{shape}[k]$, the total element count that `data` must have.

For a mini-batch of B MNIST images kept in their native two-dimensional form, `shape` $= [B, 28, 28]$: the buffer is interpreted as B consecutive blocks of 28 rows of 28 pixels each, with `data.len()` $= B \cdot 28 \cdot 28$. If instead the same batch is flattened for a fully-connected input layer, `shape` $= [B, 784]$ describes the identical buffer, reinterpreted as B rows of 784 features, same n, same underlying bytes, different logical grouping. This is exactly the "interpretation layer" role anticipated above.

The reasoning of Section 2.2.3 applies verbatim to `shape` itself: both its length (the rank, which differs between a fully-connected tensor of shape $[B, 784]$ and a convolutional one of shape $[B, C, H, W]$) and its entries (starting with B) are run-time quantities, so `shape` must equally be a `Vec<usize>` rather than a fixed-size array.

2.2.5 The `strides` Field

The remaining piece of metadata is `strides`. Given a tensor's logical `shape` (for example, a 3×4 matrix), the `strides` tell us exactly how many positions we need to skip in the underlying flat, one-dimensional memory buffer to move by one step along a specific logical axis (e.g., moving to the next row or the next column).

Because the offset formula treats `shape` and `strides` as independent inputs, any operation that only rearranges *which* multi-index maps to *which* offset, permuting axes, slicing a sub-range, reversing an axis, can be realized by rewriting these two integer tuples alone, in $O(1)$, with the underlying buffer untouched: the framework only changes the "reading rule", never the data. Operations that actually combine values, like elementwise arithmetic or matrix multiplication, still require a full pass over the buffer, but the compute kernel reads it correctly regardless of how it has been reshaped, because it consults `strides` rather than assuming a fixed layout.

For Instance, transposing a matrix means defining a new tensor T with $T[j, i] := X[i, j]$ for every admissible (i, j). this is achieved, with zero data movement, by swapping the two entries of `shape` *and*, correspondingly, the two entries of `strides`.

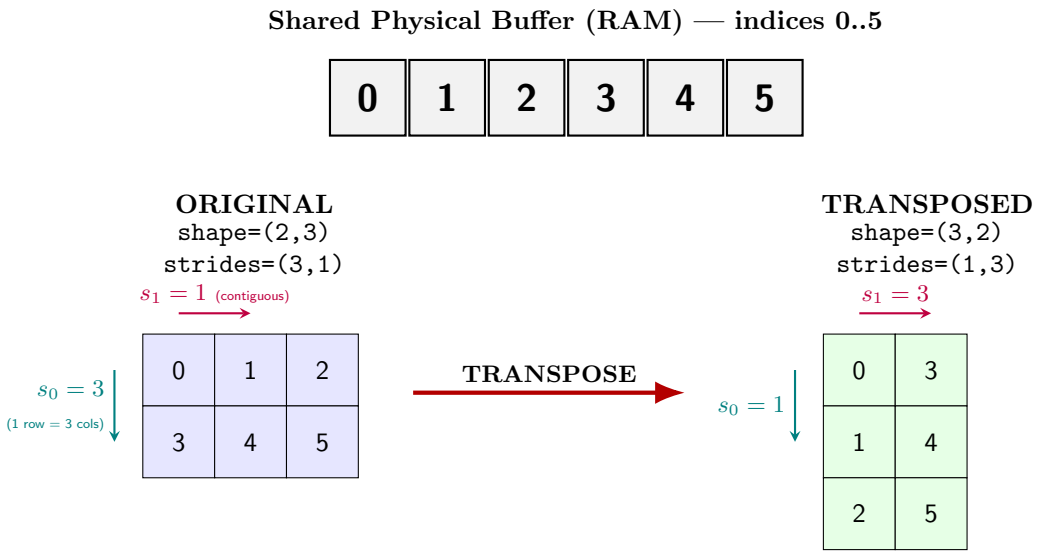

Figure 4: Same physical buffer, two different readings. On the left, advancing by one row (axis 0) skips 3 positions because each row holds 3 columns ($s_0 = 3$); advancing by one column (s_1) costs only 1 because elements within a row are contiguous. Transposing swaps `shape` and `strides`: for instance, the same buffer cell holding value 1 appears in a different position in the transposed view (row 1, column 0), with no physical data movement at all.

2.2.6 Ownership of `Tensor` Across `forward`, `backward`, and the Parameter Accessors

`Module` exposes four points at which a `Tensor` crosses a function boundary: the input of `forward`, the input of `backward`, and the two accessors `params` and `grads`.

`forward`: Ownership Transfer as a Structural Necessity In our architecture, a `Module` is responsible for saving its input during the forward pass so it can be used later in the backward pass. This creates a strict memory lifecycle requirement that makes passing `Tensor` by value the natural and most efficient design choice for this architecture.

If `forward` took its input by reference (`&Tensor`), the layer would need that reference to remain valid until a later call to `backward`. Since the activation is owned by the caller and typically discarded as execution progresses through the network, such a borrow cannot be retained safely. To preserve the activation, the layer would therefore be forced to create its own owned copy:

```
// Bad: Borrowing forces a physical clone to satisfy the lifetime requirements.
fn forward(&mut self, input: &Tensor) -> Tensor {
    self.input_cache = Some(input.clone());   // O(n) buffer duplication!
    input.dot(&self.weight.t()) + &self.bias
}
```

Taking `input: Tensor` by value solves this entirely. Along a linear chain, ownership of an activation naturally flows from one layer to the next. By transferring ownership (`move`), the layer takes full control of the tensor and caches it with zero data duplication. The tensor then remains alive exactly as long as the layer needs it.

```
// Good: Moving ownership allows zero-cost caching.
fn forward(&mut self, input: Tensor) -> Tensor {
    self.input_cache = Some(input);                    // Ownership transferred, zero clone
    let cached = self.input_cache.as_ref().unwrap(); // Read back via borrow
    cached.dot(&self.weight.t()) + &self.bias
}
```

This `move` semantics perfectly matches a linear, single-consumer topology. However, if the network contains branching (e.g., skip connections where an activation feeds multiple downstream layers), a single value cannot be moved twice. In these cases, shared ownership via `Rc<Tensor>` becomes necessary, allowing multiple consumers to access the same tensor while only incrementing a reference count rather than duplicating the underlying buffer:

```
// Shared ownership: Necessary only for branching architectures.
fn forward(&mut self, input: Rc<Tensor>) -> Tensor {
    let out = input.dot(&self.weight.t()) + &self.bias;
    self.input_cache = Some(Rc::clone(&input));  // Clones the pointer, not the buffer
    out
}
```

While `Rc<Tensor>` handles branching naturally, making it the default everywhere would introduce unnecessary pointer indirection and reference-counting overhead in the common sequential case.
For standard feed-forward chains, plain `move` semantics remain both the simplest and the most efficient choice.

`backward`: Ownership as a Design Choice While `forward` must take ownership to cache its input, `backward` has no such structural requirement. The incoming gradient (`grad_output`) is used during the operation and is not stored inside `self`.

```
fn backward(&mut self, grad_output: Tensor) -> Tensor {
    let input = self.input_cache.as_ref().unwrap(); // z_{l-1}, cached during forward

    // Gradients are accumulated rather than overwritten
    self.grad_weight += grad_output.t().dot(input);
    self.grad_bias += grad_output.sum_axis(Axis(0)).insert_axis(Axis(0));

    grad_output.dot(&self.weight) // Last use of grad_output
}
```

Notice that the parameter gradients are strictly accumulated (`+=`) rather than overwritten. While the operational details of this design will be formalized in the context of the `Optimizer` abstraction, this additive semantic is a structural necessity: it permits the accumulation of gradients across multiple independent forward-backward passes before a parameter update is enacted. Consequently, this architectural choice implicitly mandates a dedicated mechanism to zero the gradient buffers before starting a new accumulation cycle.

Because `grad_output` does not need to outlive the method call, taking a reference (`&Tensor`) would compile perfectly without forcing any $O(n)$ memory clones:

```
// This borrowing loop type-checks effortlessly without clones:
let mut a = seed_from_loss;
for layer in layers.iter_mut().rev() {
a = layer.backward(&a); // Old 'a' is dropped, no clones forced
}
```

Passing `grad_output: Tensor` by value is therefore a deliberate design choice, driven by two key advantages rather than a compiler constraint:

- **API Uniformity:** It maintains a single, consistent ownership paradigm across the entire `Module` trait, avoiding unnecessary asymmetry between `forward` and `backward`.
- **Explicit Consumption Semantics:** It makes explicit that the backward operation consumes its incoming gradient. This is particularly natural in non-linear architectures, where the execution engine may first accumulate gradients from multiple paths into a single tensor before passing that tensor to the corresponding upstream step.

`params` and `grads`: Borrowing Persistent State Unlike intermediate activations, which flow through the network and are discarded, a layer's parameters (θ_l) are persistent state. They belong to the `Module` and must remain available across the entire training lifecycle.

This changes how ownership must be handled. An accessor cannot simply move the parameters out of the module: doing so would transfer ownership and leave the module without its own weights and biases. Since `Tensor` is not `Copy`, returning the existing tensors by value would therefore move them rather than implicitly duplicate them. Cloning could produce owned copies, but would unnecessarily duplicate the parameter buffers.

The appropriate abstraction is therefore to **lend** the existing tensors through references. Different operations require different kinds of access, which leads to three accessors:

```
fn params(&self) -> Vec<&Tensor> {
vec![&self.weight, &self.bias]
}

fn params_mut(&mut self) -> Vec<&mut Tensor> {
vec![&mut self.weight, &mut self.bias]
}

fn grads(&self) -> Vec<&Tensor> {
vec![&self.grad_weight, &self.grad_bias]
}
```

Each accessor corresponds to a different requirement:

- **The Inspection Loan (`params`):** Returns immutable references (`&Tensor`) for operations such as checkpointing or logging.
- **The Update Loan (`params_mut`):** Returns mutable references (`&mut Tensor`) so an optimizer can update parameters in place ($\theta_l \leftarrow \theta_l - \eta \nabla_{\theta_l} \mathcal{L}$).
- **The Gradient Loan (`grads`):** Returns immutable references to the stored gradients, which the optimizer reads to compute the update.

The distinction between immutable and mutable loans also matters when the optimizer performs an update. Although parameters and gradients are different fields, the methods expose them through the same `Module` object: `grads(&self)` creates a shared borrow of the module, while `params_mut(&mut self)` requires an exclusive borrow. These borrows therefore cannot overlap.

The optimizer solves this by separating the two phases in time.
It first copies the gradients into owned tensors, ending the immutable borrow, and only then requests mutable access to the parameters:

```
impl SGD {
    fn step(&self, module: &mut dyn Module) {
        // 1. Immutable borrow: copy gradients into owned tensors
        let grads: Vec<Tensor> = module.grads()
            .into_iter()
            .map(|g| g.clone())
            .collect();

        // 2. Mutable borrow: update parameters
        for (theta, grad) in module.params_mut().into_iter().zip(grads.iter()) {
            *theta -= &(grad * self.lr); // In-place update
        }
    }
}
```

The clone is therefore not required by `grads()` itself.
It is needed here to create owned gradient values so that the immutable borrow can end before the mutable borrow required by `params_mut()` begins.

2.3 Design of the `Loss` Abstraction

While tensors flow uniformly through network layers, loss functions introduce a divergence: the prediction z_L is always a `Tensor`, but the target y depends on the objective. Classification targets are naturally discrete class indices, whereas regression targets are continuous quantities matching z_L.

This section assumes a single-device, CPU-only setting where `Tensor` is a dense, `f32`-only container (Section 2.2.2); device placement and integer-typed tensors are orthogonal concerns. Throughout, N denotes the number of examples in a batch and C the number of classes, matching the convention used in Figure 5; for a single example the corresponding costs are $O(1)$ and $O(C)$ respectively.

Enforcing a single concrete target type creates an architectural compromise:

- **Fixing the target as `&[usize]`** handles classification naturally, but makes regression inexpressible since continuous values lack a valid index encoding.
- **Fixing the target as `&Tensor`** handles regression and can also represent classification targets, but loses the type-level distinction between discrete labels and continuous values. Storing indices as `f32` therefore erases their semantic distinction: the type no longer expresses that a value is a nominal class label rather than an ordinary continuous quantity, potentially permitting inappropriate metric operations (such as interpolation or distance computations), depending on which operations are exposed on `Tensor`. Alternatively, a dense one-hot encoding removes the ambiguity of mapping a class to a single scalar index, but the target is still statically just a `Tensor` of `f32`, so the type itself does not communicate that it represents a one-hot classification target rather than an arbitrary continuous quantity; it also increases the target representation from $O(N)$ to $O(N \times C)$, introducing unnecessary memory and bandwidth overhead.

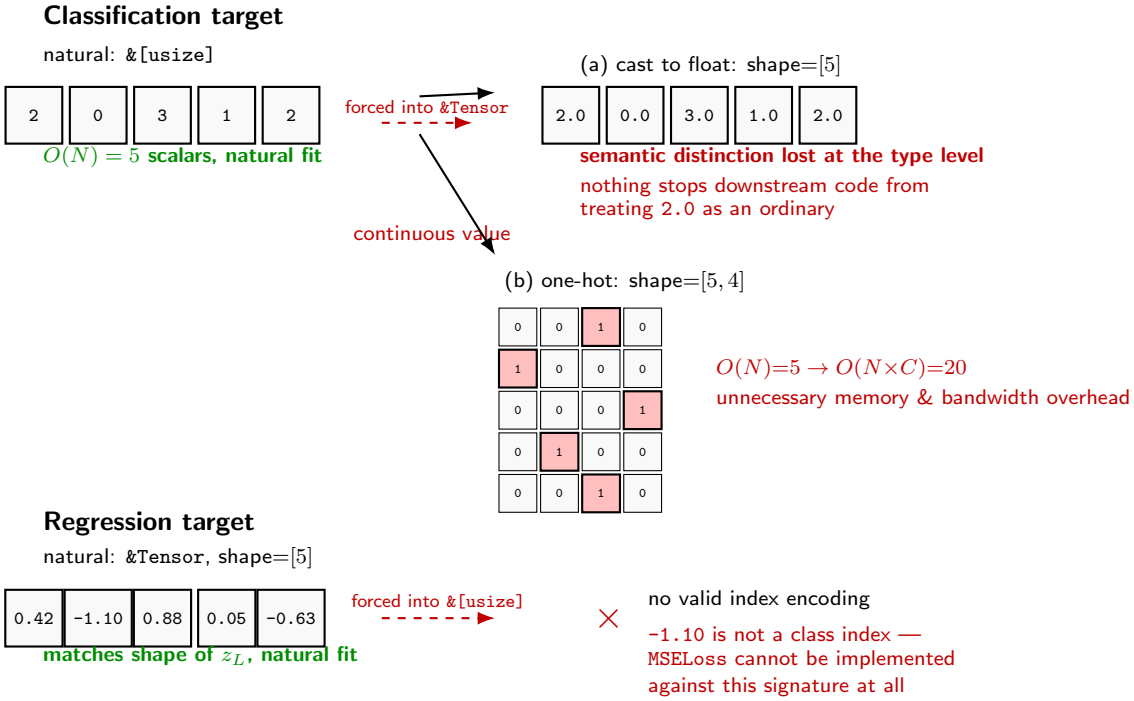

Figure 5: Forcing a single concrete target type produces asymmetric failure modes, with N examples and C classes. **Top:** fixing the target as `&Tensor` forces classification labels through one of two broken encodings — (a) casting indices to `f32`, which loses the type-level distinction between discrete labels and continuous values and can allow downstream code to misapply metric operations; or (b) a dense one-hot encoding, which grows the target representation from $O(N)$ to $O(N \times C)$ and introduces unnecessary memory and bandwidth overhead, without itself making the target type statically distinguishable from an arbitrary `Tensor`. **Bottom:** fixing the target as `&[usize]` makes regression targets inexpressible.

Although the target representation is inherently tied to the semantics of the loss, in principle the target interface could be designed around a single `Tensor` type, given suitable conventions or encoding metadata. However, the choice to adopt differentiated targets connects directly to the polymorphism taxonomy discussed earlier: rather than relying on universal parametric polymorphism, which would require explicitly threading a generic parameter through the entire call chain, we exploit Rust's trait-based polymorphism, together with **associated types**, which allow each implementation to specify its own corresponding target type. In particular, the caller does not need to propagate `Target` as a separate generic parameter through the call chain; the associated type is fixed by the concrete type

implementing `Loss` rather than being an independent parameter of the trait or a property of any particular runtime value, so the target type remains part of the `Loss` contract itself.

This mechanism is conceptually reminiscent of Haskell's type classes with associated type families, although the two systems differ in their type-inference and coherence rules. The analogy is useful insofar as both allow an interface to associate a type with a particular implementation, while preserving static safety and architectural clarity.

Each `Loss` implementation declares its target type directly within the trait definition:

```
pub trait Loss {
type Target: ?Sized;
fn forward(&self, z_l: &Tensor, target: &Self::Target) -> (f32, Tensor);
}

// Classification: accepts a slice of discrete class labels
impl Loss for SoftmaxCrossEntropy {
type Target = [usize];

fn forward(&self, z_l: &Tensor, target: &Self::Target) -> (f32, Tensor) {
// ...
}


}

// Regression: accepts a continuous activation tensor
impl Loss for MSELoss {
type Target = Tensor;

fn forward(&self, z_l: &Tensor, target: &Self::Target) -> (f32, Tensor) {
// ...
}


}
```

The `?Sized` bound allows the associated type to be dynamically sized (like the slice `[usize]`), while supporting sized types like `Tensor`.

The pair returned by `forward` is $(\ell, \nabla_{z_L}\ell)$: the scalar loss alongside its gradient with respect to the prediction `z_l`. Crucially, both quantities are computed as a mean over the N examples in the batch (i.e., inherently scaled by $1/N$). As we will see, delegating this reduction entirely to the `Loss` abstraction guarantees that the constant factor propagates linearly through the backward pass, allowing upstream components to remain completely agnostic to the batch dimension.

Finally, unlike `Module::forward` (Section 2.2.6), which consumes inputs by value to cache them for backpropagation, `Loss::forward` takes arguments by reference. In this design, the loss computes both ℓ and $\nabla_{z_L}\ell$ in a single call, so no loss-specific forward state needs to be retained for a later backward traversal. Borrowing is therefore the clean, expected choice for this API; a design that instead separated forward evaluation from a later backward pass would need to retain such state, but that is not the case here.

2.4 Design of the `Optimizer` Abstraction

Having settled `Module`, `Tensor`, and `Loss`, we turn to the fourth and last abstraction of Table 1: `Optimizer`. Its listed duties translate into four design questions:

- how much of the model the optimizer is allowed to know
- how the per-example gradient becomes a batch gradient
- how the mathematical update rule becomes code that mutates `Tensor` state in place
- how, if at all, an optimizer remembers anything from one call to the next

to which a fifth, more operational, question must be added: how these calls are sequenced inside a training loop without violating Rust's ownership rules.

How much of the model is the optimizer allowed to know? To answer the first question, the core principle is that the optimizer (such as SGD) does not know and does not need to know the specific type of layer it is modifying, whether it is a linear layer, a convolution, or an activation. To it, only the raw numbers of the weights (θ) and their gradients exist.

The mechanism in Rust achieves this through dynamic polymorphism. Instead of binding itself to a concrete model type, the `step` function accepts a polymorphic trait object, `&mut dyn Module`, which allows the same optimizer struct to accept any object implementing the `Module` trait:

```
pub trait Optimizer {
    fn step(&mut self, module: &mut dyn Module);
}
```

The practical advantage is that the exact same optimization code works without modification, whether you are training a single isolated layer or an entire, complex, nested `Sequential` network. Through this design, the optimizer is completely decoupled from the underlying architecture, avoiding the rigid coupling that a generic type parameter like `Optimizer<M: Module>` would have introduced.

How does the per-example gradient become a batch gradient? Regarding the second question, Table 1 assigns the `Optimizer` the duty of computing the batch gradient: $\nabla_{\theta_l}\mathcal{L}_{\mathcal{B}} = \frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}\nabla_{\theta_l}\ell_i$. In practice, this reduction is almost entirely handled before the optimizer even accesses the gradients.

First, the **summation** over the batch is a free by-product of matrix algebra. Because the `Tensor` shape includes the batch axis B (Section 2.2.4), the backward pass computation (e.g., `grad_output.t().dot(input)` in a linear layer) naturally contracts over this axis. Consequently, a single call to `backward` on a batch automatically yields the sum of the gradients, not $|\mathcal{B}|$ separate tensors.

Second, the **division** by $|\mathcal{B}|$ to obtain the mean is delegated to the `Loss` abstraction rather than the optimizer. Because the VJP is a linear operation (Section 2.2.1), scaling the initial backward seed $a_L = \nabla_{z_L}\ell$ by $1/|\mathcal{B}|$ propagates this exact constant factor unaltered through every subsequent VJP_{g_l}. Thus, the gradients exposed by `grads()` are already correctly averaged, allowing the `Optimizer` to remain completely unaware of the batch dimension.

The "if necessary" clause of Table 1 covers the remaining scenario: **gradient accumulation**. If a logical batch $\mathcal{B}$ is too large to fit in memory, it must be split into smaller micro-batches. To accumulate gradients across multiple forward/backward passes before a single optimization `step` is taken, the layer's `backward` method must add to the existing gradients (e.g., `self.grad_weight += ...`) rather than overwriting them. This design choice strictly necessitates an explicit `zero_grad` pass to clear the accumulated buffers before starting the next batch cycle.

How does the mathematical update rule become code that mutates `Tensor` state in place? Addressing the third question, with $(\theta_l, \nabla_{\theta_l}\mathcal{L}_{\mathcal{B}})$ available for every l, translating an update rule such as plain SGD, $\theta_l \leftarrow \theta_l - \eta\nabla_{\theta_l}\mathcal{L}_{\mathcal{B}}$, into code is a direct application of the `params_mut`/`grads` pair of Section 2.2.6:

```
pub struct Sgd { lr: f32 }

impl Optimizer for Sgd {
    fn step(&mut self, module: &mut dyn Module) {
        let grads: Vec<Tensor> = module.grads().into_iter().cloned().collect();
        for (theta, grad) in module.params_mut().into_iter().zip(grads.iter()) {
            *theta -= &(grad * self.lr); // in-place update
        }
    }
}
```

As established in Section 2.2.6, the intermediate `.cloned().collect()` is not incidental: it closes the shared borrow opened by `grads(&self)` before the exclusive borrow required by `params_mut(&mut self)` is requested, since the two cannot overlap on the same `module`.

How does an optimizer remember anything from one call to the next? To answer the fourth question, we must look at auxiliary state. While standard SGD is stateless and requires only the current weights and gradients ($s_l = \emptyset$), more advanced algorithms like Momentum require an auxiliary state. Specifically, Momentum relies on a velocity term $v_l \leftarrow \beta v_l + \nabla_{\theta_l} \mathcal{L}_{\mathcal{B}}$ that must be updated at every step and remembered for the next ($s_l = v_l \neq \emptyset$).

Because optimization algorithms and their internal auxiliary states are fundamentally decoupled from the forward and backward mechanics of neural network layers, this historical data cannot reside within the `Module`.
A layer is structurally agnostic to how it is optimized—requiring no auxiliary state for plain SGD, a velocity term for Momentum, or multiple moment buffers for more complex methods like Adam. Consequently, this state must reside within the `Optimizer` itself, stored as an internal, positionally-indexed `Vec` with one velocity tensor per parameter block. Furthermore, since the optimizer does not know the network's architecture ahead of time, this state is lazily allocated during the very first training step:

```
pub struct Momentum {
    lr: f32,
    beta: f32,
    velocity: Vec<Tensor>, // s_l, l = 1..L; empty until first step
}

impl Optimizer for Momentum {
    fn step(&mut self, module: &mut dyn Module) {
        let grads: Vec<Tensor> = module.grads().into_iter().cloned().collect();

        // Lazy allocation: initialized only on the first step
        if self.velocity.is_empty() {
            self.velocity = grads.iter().map(Tensor::zeros_like).collect();
        }

        // Zipping parameters, gradients, and velocities by position
        for ((theta, grad), v) in module.params_mut().into_iter()
            .zip(grads.iter())
            .zip(self.velocity.iter_mut())
        {
            *v = &(&*v * self.beta) + grad;
            *theta -= &(&*v * self.lr);
        }
    }
}
```

Because `zip` matches elements strictly by their index position (pairing the i-th parameter with the i-th gradient and i-th velocity), the update loop is entirely blind to the actual identity or layer-ownership of the tensors. It relies on absolute positional alignment across three independent collections: the mutable parameters returned by `params_mut()`, the cloned gradients in `grads`, and the historical states stored in `self.velocity`.

This introduces a strict operational invariant: `params_mut()` and `grads()` must enumerate the parameter blocks in the exact same deterministic order on every single training step. If a custom module implementation were to alter its traversal order between calls, `zip` would silently mispair updates—applying a velocity tensor belonging to one layer's weight matrix to an entirely different layer's bias vector, corrupting training without triggering any runtime panic. While our `Sequential` container (Section 2.1) naturally upholds this invariant by iterating over its layers in a fixed order, the `Module` trait signature cannot mathematically enforce it. It remains a structural convention—a functional contract between `Module` and `Optimizer` that must be respected by the developer, as it is not statically verified by Rust's type system.

How are these calls sequenced inside a training loop without violating Rust's ownership rules? Finally, to answer the fifth, more operational question, we must consider the training lifecycle. The reduction in Section 2.4 assumed `backward` *accumulates* into `grad_weight` (`self.grad_weight += ...`) rather than overwriting it as originally shown. This one-word change has a consequence: without an explicit reset, gradients from a previous step would silently add onto the next one. Symmetrically to how `params/grads/params_mut` were justified in Section 2.2.6 by parameters being persistent state *owned* by the module, resetting that same state is the module's responsibility, not the optimizer's — `Optimizer` only ever reads gradients, it never has a reason to clear them:

```
pub trait Module {
```

```
    // ...forward, backward, params, params_mut, grads as before...
    fn zero_grad(&mut self);
}

impl Module for Sequential {
    fn zero_grad(&mut self) {
        for layer in self.layers.iter_mut() { layer.zero_grad(); }
    }
}
```

The training loop is then a strict sequence of three phases, none of which may overlap in time:

```
model.zero_grad();
for micro_batch in batch.chunks(k) {                    // 1..K micro-batches
    let z_l = model.forward(micro_batch.input);
    let (_, grad_out) = loss_fn.forward(&z_l, &micro_batch.target);
    model.backward(grad_out);                           // accumulates into grad_*
}
optimizer.step(&mut model);                              // consumes and updates
```

This strict ordering is not merely good practice: `model.forward`/`model.backward` and `optimizer.step` each require an exclusive `&mut` borrow of `model`, so the borrow checker itself would reject any attempt to interleave them. The same constraint that, in Section 2.2.6, forced the gradient-then-parameter borrows inside a single `step` to be sequenced rather than simultaneous, here forces the entire lifecycle, reset, accumulate, update, into non-overlapping phases at the scale of the whole training loop.

Advanced Refinements Two common refinements confirm, rather than complicate, the decoupling discussed in our first question (Section 2.4): both are implemented entirely inside `Optimizer::step`, with no change whatsoever to `Module` or `Tensor`.

Weight decay. L2 regularization modifies the update rule to $\theta_l \leftarrow \theta_l - \eta(\nabla_{\theta_l}\mathcal{L}_\mathcal{B} + \lambda\theta_l)$. Since `params_mut()` already exposes θ_l itself, not only its gradient, the extra term is a purely local addition to `step`:

```
for (theta, grad) in module.params_mut().into_iter().zip(grads.iter()) {
    let decayed = grad + &(&*theta * self.weight_decay);
    *theta -= &(decayed * self.lr);
}
```

Gradient clipping. To guard against numerically exploding gradients, the global norm $\|\nabla\mathcal{L}_\mathcal{B}\| = \left(\sum_l \|\nabla_{\theta_l}\mathcal{L}_\mathcal{B}\|^2\right)^{1/2}$ is computed over the already-owned `grads: Vec<Tensor>` local copy — the same clone introduced in Section 2.4 to release the borrow on `module`, and, if it exceeds a threshold, every gradient is rescaled before being consumed by the update rule:

```
fn clip_grad_norm(grads: &mut [Tensor], max_norm: f32) {
    let total_norm = grads.iter().map(Tensor::squared_sum).sum::<f32>().sqrt();
    if total_norm > max_norm {
        let scale = max_norm / (total_norm + 1e-6);
        for g in grads.iter_mut() { *g *= scale; }
    }
}
```

In both cases the pre-existing separation between an owned, mutable copy of the gradients and the borrowed parameters, already necessary purely to satisfy the borrow checker, turns out to be exactly the extension point these two refinements need.

3 How I Used AI?

Throughout this project, I utilized various AI tools—specifically the free web interfaces of ChatGPT, Claude, and Gemini. I deliberately avoided integrated IDE agents like Claude Code or Codex.

The guiding principle behind my workflow was absolute ownership: I wanted to maintain total control over both architectural abstractions and implementation details. Consequently, I treated the AI not as an autonomous developer to which I could outsource large modules, but rather as an interactive sounding board. My approach was iterative, but I explicitly avoided the pattern of delegating a massive task and then endlessly tweaking a bloated, AI-generated output.

3.1 The Trade-off: Speed vs. Comprehension

Reflecting on this process, I have come to a stark realization regarding the current state of AI. Even using only the free-tier models, the AI could likely have completed this entire project in a fraction of the time, if I had simply allowed it to take the wheel. However, doing so requires a fundamental sacrifice: abandoning total control over complexity and deep personal comprehension.

I am unsure what the future holds for developers, but blindly delegating logic to an AI and attempting to reverse-engineer its thought process post-factum is a massive cognitive burden—one that current AI models are not quite capable of helping us navigate yet. For repetitive, boilerplate, or non-critical tasks, sacrificing control for speed makes sense. But for the core challenges of this project, such as designing a minimal, interconnected library from scratch, delegation was not an option. Outsourcing even a single central component would have compromised my understanding of the entire system.

Where I *did* use the AI aggressively was in the mere act of typing. Once I had solidified my ideas and architectural choices, I used the AI as a highly advanced compiler to translate my logic into idiomatic Rust code. For this purely mechanical translation, it was exceptionally effective.

3.2 A valid self-critique

In hindsight, a valid self-critique is that I may have gripped the steering wheel a bit too tightly. The future of software engineering likely lies in finding a balanced synergy with AI. My rigid desire for total control is heavily biased by my professional background: having previously worked in defense software, absolute architectural and implementational certainty was not just a preference, but a strict requirement.

3.3 Rapid Voice Prompting

Beyond coding, I frequently use AI to study. During this project, I experienced firsthand that seeking deep understanding does not necessarily require the most powerful models or massive, comprehensive outputs.

In fact, my most effective learning occurred when I used faster, slightly weaker models via **voice commands** (speech-to-text, not the continuous audio-chat mode). I intentionally instructed the AI to give me extremely brief, punchy answers. This allowed me to rapidly read the response and immediately fire back with my doubts, creating a fast-paced, Socratic ping-pong. By forcing the AI to take small steps with me, I converged on solutions that I truly understood.

I believe this worked because the reasoning process remained primarily mine. Rather than passively consuming long explanations, I continuously directed the conversation toward the specific points I wanted to clarify. The AI acted more like an interactive interlocutor than a lecturer: it provided hints, alternative perspectives, and small pieces of information, while I actively determined the path of inquiry. In this setting, the quality of the learning process depended less on the model producing a complete answer and more on my ability to ask targeted questions and iteratively refine my understanding. As a consequence, even less capable models often proved sufficient, since their role was not to solve the problem for me but to support an ongoing dialogue in which I remained the primary agent of reasoning.

Example: Brainstorming the Autograd Backward Pass

The following is a condensed transcript of a real voice-command session. The casual, halting nature of the prompts is preserved to show how raw spoken thoughts were refined into architectural abstractions.

Me (Voice): Alright, so for now I have this. If someone asks, "How do you implement the backward pass?" I'd say I need two things. I need a mechanism to calculate the local step, and I need to know the actual *order* of steps to follow. But wait... couldn't someone argue that the Forward pass already gives me the order? Since we have a finite set of mechanisms translating these transformations... I don't really need a new abstraction for the

order, right?

AI: You're right, the order is given by the Forward pass, but there's a conceptual trap! The Chain Rule dictates we go in reverse: $g_L \to g_{L-1} \to \cdots \to g_1$. But unlike the Forward pass, the Backward pass requires *memory*. You need the intermediate states (x) calculated during the Forward pass to compute the local gradients. You need a way to record the history.

Me (Voice): No, wait, calm down a second. I don't care about the mathematical derivatives right now. I want to build my mental framework strictly through abstractions. Let's stay concrete. I need exactly two things: to know *which* steps to do, and to know *how* to do each step. That's it, right?

AI: You hit the nail on the head. Architecturally, you only need two things: 1. **The Local Step Abstraction:** How to take a gradient from the "future" and pass it to the "past". 2. **The Flow Coordinator:** A mechanism that knows the topology and triggers these steps in reverse order.

Me (Voice): Okay, keeping it concrete. To know *which* steps to do, if the user explicitly defined the order using our "Composable Transformations" abstraction... doesn't that leave a trace? I'm getting lost. What even *is* that abstraction at runtime?

AI: A "Composable Transformation" is just an interface—a base class with a `forward()` method. By itself, it computes math and immediately forgets the past. To leave a trace for the Backward pass, it must actively record its execution onto a Tape or a Dynamic Graph.

Me (Voice): Stop. You need to be much, much shorter. This is a back-and-forth conversation. Keep it brief, always. Okay, so my question: Is the "Composable Transformation Abstraction" essentially just the implementation of a single Layer? Because I called it a "finite set of mechanisms" to keep it abstract.

AI: Yes, exactly. In code, it is simply the `Layer` or `Module` interface. The "finite set" just means your basic building blocks (Linear, ReLU, etc.).

Me (Voice): Okay! So now, continuing my logic: if I build an architecture out of this finite set of layers, the architecture itself acts as a map. I can just extract the backwards order directly from the architecture's representation, right?

AI: Yes, but with one distinction: If your graph is **static** (a fixed list of layers), the architecture *is* the map. If your graph is **dynamic** (using `if` statements or loops in the forward pass), the architecture isn't enough; the map is only created dynamically at runtime by recording the trace.

Me (Voice): Perfect. That distinction is exactly what I needed to design the coordinator...

This rapid, constrained back-and-forth was the exact methodology I applied across multiple complex sections of the program, proving that AI is often most powerful when heavily restricted by the user.

3.4 Cross-Checking Methodology: Validating the AI output

A natural question arises: how did I cross-check the AI's outputs to ensure correctness?
The validation strategy depended entirely on whether I was using the AI for high-level reasoning, mathematical derivations, visual generation, or mechanical code translation.

For the reasoning and brainstorming phases, I did *not* perform continuous, step-by-step cross-checking. Stopping to validate every micro-turn would have shattered the cognitive flow, particularly since I was the one steering the direction of the conversation. Instead, I built the logic iteratively, tracking my thoughts on paper. Once a complete line of reasoning was finished, and I felt reasonably confident in it, I performed a single, consolidated cross-check. I would take the final conclusion to a fresh chat or even an entirely different AI model, specifically prompting it to check for availability bias inherited from the previous session.

Most of the time, no hidden flaws emerged because the framework was already structurally sound; I had built it myself with the AI acting merely as a scribe. Interrupted cross-checking mid-stream would have been entirely counterproductive.

When dealing with theoretical foundations tied directly to the curriculum, such as the discussion on polymorphism early in Chapter 2, my cross-checking relied strictly on the official course materials. To generate this section, I provided the AI with a highly targeted prompt to ensure academic rigor:

> *"Act as an expert in programming language theory. Synthesize a formal explanation of polymorphism, explicitly distinguishing between ad hoc (overriding, overloading) and universal polymorphism. Contrast traditional class-based object-oriented approaches with trait-based paradigms (e.g., Rust). Conclude by*

framing universal polymorphism as the necessary theoretical foundation for achieving structural uniformity across neural network layers."

Once the AI generated the draft, I manually cross-referenced its definitions and taxonomy directly against the course lecture notes and slides, ensuring strict alignment with the terminology taught in class.

For mathematical derivations, specifically the backpropagation strategy and the forward/backward passes detailed in the appendix, a different twofold validation was required. First, I passed the AI-generated derivations to a secondary LLM specifically tasked with auditing for subscript and indexing typos. Second, and more importantly, I performed a manual sanity check by explicitly tracking the dimensions of all matrices and vectors involved at each step, guaranteeing that every dot product and element-wise operation was algebraically valid.

The generation of visual assets using TikZ required an iterative validation approach. The AI occasionally generated code that resulted in overlapping nodes or misaligned geometries. To validate and correct these, I manually compiled and inspected the renders. When a diagram lacked clarity, I sent the TikZ code to a different AI model alongside the already-validated text paragraph it was meant to illustrate. I instructed this secondary AI to reorganize the spatial layout to make it highly "pedagogical and didactic," ensuring the visual topology perfectly mirrored the logic of the accompanying text.

Finally, when it came to code generation, the approach shifted to strict granular review. For translating the conceptual framework into actual Rust code, I relied on a manual, rigorous cross-check. I personally reviewed the final outputs line by line to ensure the generated code faithfully respected the architectural choices we had meticulously hammered out during the Socratic sessions.